

Team Assignment: CBT Mock Test Platform MVP

Time Limit: 7 – 10 Days

1. Objective

Build and deploy a working MVP (Minimum Viable Product) of a secure Computer-Based Test (CBT) / Mock Test Platform within 7 to 10 days.

The purpose of this assignment is to evaluate your:

- Full-stack development skills
- Backend and database design
- Authentication and authorization
- Security awareness
- Exam-state management
- Evaluation/grading logic
- Problem-solving and engineering decisions

Note: AI coding tools are allowed. However, you should understand and be able to explain the implementation you submit.

2. Modules

The system should contain the following three major modules. You are free to decide how these modules are designed and implemented, including the architecture, database schema, APIs, frontend structure, and technology stack.

- **Module 1 — Admin & Test Management:** Admin should be able to create and manage mock tests and questions.
- **Module 2 — Student CBT Examination:** Students should be able to select a test, read instructions, take the examination, and submit it.
- **Module 3 — Security, Evaluation & Analytics:** The system should provide exam security, server-side evaluation, result generation, and basic reporting.

3. Module 1 — Admin & Test Management

The admin portal should provide:

- Secure admin login
- Admin authorization/RBAC (Role-Based Access Control)
- Create, edit, view, and delete mock tests
- Activate/deactivate tests
- Configure: Subject, Topic, Language, Exam duration, Marking scheme
- Create, edit, and delete questions

Question Support:

- **MCQ** (Multiple Choice Questions)
- **MSQ** (Multiple Select Questions) (Optional)
- **NAT** (Numerical Answer Type) (Optional)
- *Note: Questions should support basic text formatting and images/diagrams.*

Default Test Configuration (for the MVP):

- **Exam duration:** 15 minutes
- **Questions:** 8–10 per mock test
- **Marking Scheme:**
 - MCQ: +4 correct, -1 incorrect, 0 unattempted
 - MSQ: +4 correct, 0 incorrect (Optional)
 - NAT: +4 correct, 0 incorrect (Optional)
- *Requirement: The marking configuration should be associated with the test/question and should not depend only on frontend logic.*

4. Module 2 — Student CBT Examination

The student flow should include:

Login → Test Selection → Instructions → Start Exam → Examination → Submission → Result

Test Selection

Students should be able to find/select tests using: Subject, Topic, and Language.

Pre-Exam Instructions

The instruction screen should clearly communicate the examination rules, including:

- Examination duration
- Server-controlled countdown timer
- Automatic submission when time expires
- Two-panel examination layout
- Question Palette & Question status indicators
- Calculator availability
- MCQ/MSQ/NAT behavior
- Marking scheme
- Security restrictions

5. CBT Examination Interface

The examination interface should provide:

Question Panel

- Display one question at a time.
- Support MCQ, MSQ, and NAT.

- Display options appropriately:
 - **MCQ:** Circular controls (Radio buttons)
 - **MSQ:** Square checkboxes (Optional)
 - **NAT:** Numerical input / virtual numeric keypad (Optional)
- Support expanding/collapsing the Question Panel.

Question Palette

The palette should display question status such as:

- Not Visited
- Answered
- Not Answered
- Marked for Review
- Answered & Marked for Review
- *Clicking a question number should navigate to that question.*

Required Controls

Use the following exact button names:

- **Save & Next**
- **Mark for Review & Next**
- **Clear Response**

Question Paper

Allow the candidate to view all questions together in a single view.

6. Exam Timer & State

- **Server-Controlled Timer:** The examination duration (15 mins for MVP) must be controlled by the server.
- The candidate should not be able to reset or extend the examination simply by refreshing the page or manipulating frontend state.
- **State Persistence:** The system should persist important exam state, including: Answers, Question status, Mark-for-review state, Exam start time, Exam attempt status, and Security/interruption events.
- **Auto-Submit:** When the timer reaches zero, the examination should automatically end and submit.

7. Module 3 — Security, Evaluation & Analytics

Examination Security

During the examination, implement appropriate browser-level restrictions including:

- Full-screen mode (Initiated by the "Start Exam" action)

- Detect leaving full-screen
- Detect tab/window switching
- Detect page refresh
- Disable right-click
- Disable copy/paste
- Restrict common keyboard shortcuts
- Screenshot protection (where technically possible)

System Interruption

The system must record interruptions during an examination (e.g., leaving the assessment window, switching tabs, using restricted shortcuts, refreshing, exiting full-screen).

- When an interruption occurs, the assessment should be locked/interrupted, and the event recorded against the exam attempt.
- Display a warning to the candidate explaining the recorded interruption and instructions to unlock/relaunch.
- *Browser security limitations should be documented in the README.*

8. Backend Security

Security-critical operations must be enforced on the backend.

Mandatory Requirements:

- Secure authentication & Backend authorization/RBAC
- Password hashing
- Protected admin APIs
- Input validation
- SQL/NoSQL injection protection
- XSS protection/sanitization
- Secure CORS configuration
- Basic rate limiting for sensitive endpoints
- No secrets/API keys committed to the repository

CRITICAL REQUIREMENT

- **Correct answers must NOT be sent to the frontend during an active examination.**
- The frontend must not be trusted for correct answers, final score, marking, or security-critical exam state.
- Final evaluation must happen on the backend.

9. Submission & Evaluation

Submission

Before final submission, show a confirmation summary containing:

- Total questions, Answered, Not Answered, Skipped, Marked for Review

Student Result Screen

After the backend calculates the result according to the configured marking scheme, display:

- Final Score & Total Marks
- Correct, Incorrect, and Skipped counts
- **Question-wise result:** Candidate's answer, Correct answer, Marks obtained
- *After the examination ends, exit full-screen mode and return to the normal interface.*

10. Admin Reports

The admin should be able to view basic examination results via a simple MVP dashboard, including:

- Test attempts
- Student-wise scores
- Correct/incorrect/skipped questions
- Basic question-wise performance

11. Deployment

The complete application must be deployed and accessible through a live URL using **HTTPS**. You may use any free/low-cost hosting platform (e.g., Vercel, Render, Railway, Supabase, Firebase).

12. Deliverables

Submit the following:

1. **Live Application:** Working Student and Admin portals.
2. **Source Code:** GitHub/GitLab repository containing the source code, README, setup instructions, environment variable example (.env.example), and database setup/migrations (if applicable).
3. **Test Credentials:** Admin credentials and Student credentials (or registration instructions).
4. **Technical Documentation:** A short README section covering: Architecture, Database design, Authentication, Exam-state management, Evaluation flow, Security implementation, and Known limitations.

13. Technology & Implementation

Technology selection is entirely your choice. You are free to decide the frontend, backend, database, system architecture, database schema, API structure, authentication approach, state management, component structure, and deployment architecture.

*The requirements above describe **what** the system must do, not **how** you must build it.*

14. AI Usage

AI-assisted development (ChatGPT, Claude, GitHub Copilot, Cursor, etc.) is allowed. However, you are responsible for the final implementation. During evaluation, you may be asked to explain, modify, or debug your implementation.

15. Evaluation

Weightage

Area	Weight
Core Functionality & CBT Experience	30%
Security & Backend Integrity	30%
Architecture & Code Quality	20%
Evaluation & Analytics	10%
Deployment & Documentation	10%

Evaluation Checklist

We will test the application as a real user and via API/browser-level inspection.

- ☐ Student can complete the complete exam flow
- ☐ Admin can create/manage tests and questions
- ☐ MCQ, MSQ and NAT work correctly
- ☐ Question Palette states update correctly
- ☐ Save & Next works correctly
- ☐ Mark for Review & Next works correctly
- ☐ Clear Response works correctly
- ☐ Question Paper works correctly
- ☐ Timer is controlled by the server
- ☐ Refresh cannot reset the examination
- ☐ Time expiry automatically ends the examination
- ☐ Full-screen mode works
- ☐ Tab/window switching is detected
- ☐ Full-screen exit is detected
- ☐ Refresh/interruption events are recorded
- ☐ Copy/paste restrictions work
- ☐ Common restricted shortcuts are handled

- ☐ Screenshot protection is implemented where technically feasible
- ☐ Student cannot access admin functionality
- ☐ Student cannot manipulate the final score
- ☐ Correct answers are not exposed during the active examination
- ☐ Backend performs final grading
- ☐ +4 / -1 / 0 marking works correctly
- ☐ MSQ/NAT evaluation works correctly
- ☐ Exam answers survive appropriate state interruptions
- ☐ Invalid/malicious API requests are rejected
- ☐ Authentication and authorization cannot be bypassed
- ☐ Application is deployed and accessible
- ☐ No secrets are exposed in the repository

16. Final Expectation

This is an MVP development assessment. You are **not** expected to build an enterprise-grade examination platform.

Focus on building a system that is:

Functional → Secure → Correct → Maintainable → Deployable

We will value a¹ well-engineered working MVP more than a large number of incomplete features.

¹ GeoBucket Solutions Pvt. Ltd.